JBI010 - Assignment - Student Lifestyle and Academic Performance (2025-2026)

Introduction

In today's academic landscape, student success depends on more than just classroom learning. Lifestyle factors such as study habits, exercise, and access to learning resources play a crucial role in shaping academic performance. This assignment aims to analyze the relationship between these factors and student exam scores to support evidence-based decision-making in educational contexts, such as offering personalized academic support and helping students make informed choices about their habits.

This assignment uses a dataset that captures a wide range of behavioral, academic, and socio-economic variables. These include academic major, study hours, motivation level, time management, as well as several other related factors.

Through a sequence of structured programming tasks, we identify key statistical trends to evaluate how lifestyle choices may affect academic outcomes. The approach emphasizes clean, testable Python code and sound data analysis techniques applied to real-world educational data.

Additionally, we extend our assignment by building an interactive dashboard using **Dash**, a Python framework for creating data applications. Although Dash is not formally covered in the course, students are expected to explore it independently using online resources and tutorials. This dashboard will provide a dynamic, user-friendly interface to explore exam performance across various behavioral attributes.

Grading

This assignment consists of **6 tasks,** including 1 **bonus task**, each contributing a different percentage toward the final grade. To ensure correctness, students are expected to implement **their unit tests** that thoroughly validate the functionality of their code. In addition to these, the submitted work will be evaluated against a separate set of test cases maintained by the course

staff, which are not visible to students. Note that passing both the student-written and staff-maintained test cases does not guarantee full marks; the final grade also depends on code quality, structure, and adherence to assignment guidelines.

All functions must include **type hints** and a **clear, meaningful docstring** describing their purpose. Students must also follow the **Python Coding Standard** provided on Canvas; failure to do so may result in point deductions.

The **final grade** will be determined based on multiple factors: the correctness and quality of the implementation, the completeness and relevance of the student-written tests, the results of our internal test suite, and the depth of insight in open-ended tasks involving analysis or reflection.

Use of AI Statement

You must include a clear and honest statement in your report describing how you used AI tools during the assignment. While using AI to generate partial or complete solutions is **not allowed**, you may use AI to help understand the problem or clarify concepts. You need to honestly state what you used and how, e.g., to resolve errors, understand something better. This statement is **mandatory**, and failure to provide it may result in a deduction of points from your final grade.

Tests

In the tests folder, you will find the test files, which include a few predefined test cases for Tasks 2 and 3. These examples are provided to guide your testing process, but are **not exhaustive**. You are expected to implement a comprehensive set of unit tests that thoroughly check the correctness of your implementation. **For each functionality, you should include well-designed test cases** to verify correct behavior under different conditions. Aim to cover common scenarios, edge cases, invalid inputs, and boundary conditions to ensure the robustness of your code.

In addition to your tests, your final program will be evaluated against a comprehensive internal suite of hidden test cases maintained by the course staff. These are not accessible to you, **so careful and thoughtful testing on your part is essential**.

Tasks

Each task in this assignment must be implemented within the **designated marker lines** provided in the assignment template. **Do not rename or move provided files**, as they are used for automated testing and evaluation. The Python files are organized into clearly defined sections for each task, marked with comment blocks such as:

```
#// BEGIN_TODO [Task x] Title of the task x

# Write your code here

#// END_TODO [Task x]
```

Failure to follow the file structure or marker conventions may result in a lower grade, as it can interfere with the automated grading process.

Dataset

For this assignment, we utilize a modified version of the publicly available *Student Habits and Academic Performance* dataset from Kaggle. This dataset offers a rich collection of features related to students' academic performance, daily habits, lifestyle choices, and socio-economic background. It serves as a strong foundation for exploring how different behavioral and contextual factors may influence educational performance, which is measured through exam scores.

To better match the learning objectives of this assignment, certain fields in the original dataset have been modified, removed, or renamed. The final version used in this assignment includes the following fields:

- **student_id**: A unique 6-digit number identifier for each student (e.g., 100000, 179999)
- **age**: Student's age in years (e.g., 17, 25, 32).
- **major**: Student's field of study (e.g., Computer Science, Arts, Business).
- **study_hours_per_day**: Average daily hours spent studying (0-20 scale).

- **social_media_hours**: Average daily hours on social media (0-20 scale).
- **part_time_job**: Whether the student has a part-time job (Yes, No).
- **attendance_percentage**: Percentage of classes attended (0-100).
- **sleep_hours**: Average hours of sleep per day (0-20 scale).
- **diet_quality**: Quality of the student's diet (Poor, Fair, Good).
- **exercise_frequency**: Number of physical activity sessions per week (e.g., 0, 4, 9).
- **internet_quality**: Quality of home Internet (Low, Medium, High).
- **extracurricular_participation**: Participation in extracurricular activities (Yes, No).
- **previous_gpa**: GPA from the previous academic term (0–4 scale).
- **semester**: Current academic semester (1–8 scale).
- **dropout_risk**: Estimated risk of dropping out (Yes, No).
- **social_activity**: Level of engagement in social or group activities (0 = None, 5 = Very active).
- **screen_time**: Total daily screen time in hours (0-20 scale).
- **study_environment**: Typical study setting (Learning Group, Library, Quiet Room, Dorm, Café).
- **access_to_tutoring**: Whether the student has access to tutoring (Yes, No).
- **motivation_level**: Self-assessed motivation (1–10 scale).
- **learning_style**: Preferred learning style (Reading, Visual, Auditory, Kinesthetic).
- **time_management_score**: Self-reported time management score (0–10 scale).
- **exam_score**: Final exam performance score (0–100 scale).

Task 1: Create Issues on GitLab (5 Points)

Version control systems like Git are essential for tracking changes and enabling effective collaboration, especially in team-based software projects. In this course, we use the TU/e GitLab platform to manage your project code and coordinate teamwork.

For this task, you will use GitLab's **issue tracking** feature to plan and organize your work. You and your partner will coordinate and use this service to speed up the collaboration and development process. Thus, for each Task, you need to:

a) **Create Issues:**

- Make at least **one GitLab issue per assignment Task**: Tasks 2–5.
- You're free to break tasks into smaller subtasks to support better planning and workload distribution.

b) **Use Clear Commit Messages:**

- All team members must push commits **frequently and evenly**.
- Write meaningful commit messages starting with a verb in infinitive form (e.g., *"Add function to calculate average score"*).
- You may also optionally use Conventional Commits to structure your commit messages.

c) **Close Issues When Done:**

- Close each issue when the task is completed.
- On platforms like GitLab, you can automatically close an issue by including one of the predefined keywords, such as "close," followed by the "#" symbol and the issue number (e.g., "close#10") in the commit message or merge request description. This action will close the issue automatically once the commit reaches the default branch or the merge request is merged. Alternatively, you can manually close issues through the platform's web interface.

d) **Collaborate Effectively:**

- Regularly, pull your partner's changes and push your own to avoid conflicts.
- If a conflict occurs, resolve it locally before pushing again.

Task 2: Define the Core Data Classes (35 Points)

In this task, you will design the core data structures that form the foundation for the rest of the assignment. Your goal is to apply principles of clean, reusable, and extensible object-oriented design to model both individual data records and the overall dataset.

Note: Design and implement the internal structure carefully and thoughtfully. Do not use AI-generated code to complete this task. Instead, focus on understanding and applying solid object-oriented design principles on your own.

Attribute requirements: You are responsible for identifying and defining all attributes necessary to support the specified methods' functionality. Note that this assignment does not provide any hints or guidance regarding which attributes to include. You must determine them based on the method requirements and overall design goals.

Method requirements: Each method must be implemented exactly as described, using the specified input parameters and return types to ensure compatibility with automated test cases.

Library usage: You are **only allowed to use the Python libraries that are imported in the provided project skeleton**. Do not import or use any other libraries unless explicitly permitted in the instructions. This ensures consistency and compatibility across all student submissions.

Part A: Implement the `Student` Class (15 Points)

The `Student` class models a single student's data record. It encapsulates relevant fields from the dataset (e.g., `student_ID`, `age`, `learning_style`, `exam_score`), allowing each object to be a structured, self-contained representation of an individual student's data. A Python file named `student.py` is provided. You are required to implement all of the following methods within the `Student` class in the provided Python file.

Required methods (description):

- **`__init__`**

  Initializes all attributes corresponding to the dataset's fields for a single student object.

  Parameters: Relevant attributes matching the dataset fields

  Return: None

- **`__str__`**

  Returns a string representation of the student object in the following format:

Format: StudentID: <id>, Age: <age>, Major: <major>, Exam Score: <score> Example: StudentID: 1234, Age: 20, Major: Computer, Exam Score: 87.5

- The output must match this format **exactly**, including field names, order, punctuation, and spacing. This format is required for automated testing compatibility.
- The values must reflect the actual data stored in the object.

Parameters: None

Return: Text

- `to_dict`

  Converts the student object into a dictionary where keys are attribute names and values are the corresponding attribute values.

  Parameters: None

  Return: Dictionary with the student's attributes

- `update_from_dict`

  Updates the attributes of the `Student` object based on a given dictionary. Only keys in the dictionary that match existing attribute names are used to update those attributes with the provided values. The method handles appropriate typecasting for numeric fields (e.g., converting strings to integers or floats where necessary). If a key in the dictionary does not correspond to any attribute of the `Student` object, it is ignored, and no changes are made for that key.

  Parameters: Dictionary containing attribute names as keys and new attribute values

  Return: None

Note: To ensure your implementation is correct, you must create a comprehensive test suite. A sample test file, `test_student.py`, has been provided for the `student` class.

Part B: Implement the `StudentDataset` Class (20 Points)

The `StudentDataset` class stores the complete collection of student records and is responsible for **loading** them in a structured format. It **imports** data from the CSV file, **validates** and **cleans** each record, and **creates** the corresponding `Student` objects. This forms the foundation for any further analysis or processing. A Python file named `student_dataset.py` is provided for your implementation, along with the CSV dataset located in the `data` folder. Do not move or rename the dataset file, as the automated tests depend on its location.

- **Required methods (description):**
  - **`__init__`**

    Initializes a new `StudentDataset` object with an **empty list** designed to store multiple `Student` objects.

    Parameters: None

    Return: None

  - **`load_from_csv`**

    This method reads and processes each row in the CSV file to create a `Student` object using the appropriate constructor or any helper methods that you might define. It performs typecasting for all field values (e.g., `int, float, str`) and applies strict data validation to ensure consistency and accuracy across the dataset.

    Parameters: The file path to the CSV file

    Return: None

    Notes:

    - You must validate **all** categorical fields with limited, predefined values. For example:
      - part_time_job: Valid values are only "Yes" or "No".
      - diet_quality: Valid values are "Poor", "Fair", or "Good".
      - exam_score: Must be a numeric value between 0 and 100, inclusive.
      - Other categorical fields with fixed options must be validated similarly.

      Any object containing **empty**, **invalid, or misspelled values** in such fields must be **excluded** from the dataset. However, **lowercase versions of valid values** (e.g.,

"good" instead of "Good" or "computer sciEnce" instead of "Computer Science") are acceptable. They should be **converted to the correct format,** where the first letter of each word is capitalized (e.g., "Good" or "Computer Science").

- Ensure that column headers in the CSV file are properly normalized before processing. This includes:
  - Converting all letters in column headers to lowercase (e.g., "Age" → "age" or "Part_Time_JOB" → "part_time_job").
  - Multi-word field names with a space between them must be converted to **snake_case** (e.g., "Screen Time" → "screen_time").

Implementation tip: You may implement all functionality directly inside the `load_from_csv` method, or implement some helper methods and use them inside `load_from_csv`. The second option improves clarity and reuse, which is recommended.

Robustness: The CSV files used for testing may contain **intentionally corrupted rows** (e.g., incorrect values, such as "Maybe" for part_time_job or 150 for exam_score; or incorrect column headers, like "Part Time Job" instead of "part_time_job"). Your method must detect such issues, correct them when possible, or skip invalid rows as discussed above.

Note: To ensure your implementation is correct, you must create a comprehensive test suite. A sample test file, `test_student_dataset.py`, has been provided for this class.

Task 3: Statistical Insights and Analytical Methods (30 Points)

In this task, you will implement analytical capabilities within the `PerformanceAnalyzer` class by defining methods that compute statistical insights from the dataset. These methods are intended to evaluate your understanding of foundational statistics, data structures, object-oriented programming, and clean method design. A Python file named `performance_analyzer.py` is provided for your implementation. This task builds directly on the core classes you defined in Task 2.

Library usage: You are only allowed to use the Python libraries that are imported in the provided file. Do not import or use any other libraries unless explicitly permitted in the instructions.

Part A: Attribute Filtering Utilities (10 Points)

- **Required methods (description):**
  - **`__init__`**

    Initializes a new `PerformanceAnalyzer` object with the provided dataset.

    Parameters: A `StudentDataset` object containing the student objects to be analyzed.

    Return: None

    Note: The dataset passed to this method should already be validated and loaded (e.g., using `StudentDataset.load_from_csv`). This ensures that the methods in PerformanceAnalyzer operate on clean, consistent data.

  - **`filter_by_attribute`**

    Returns a list of `Student` objects where a specified attribute matches the given value, **using case-insensitive comparison**.

    Parameters:

    - attribute: The name of the field to filter by (e.g., "major").
    - value: The string value to match against (e.g., "Computer Science").

    Return: A list of `Student` objects where the attribute matches the value

    Example: Calling `filter_by_attribute("major", "Computer Science")` returns a list of `Student` objects whose major is "Computer Science". The comparison is case-insensitive, so calling `filter_by_attribute("major", "computer science")` or `filter_by_attribute("major", "COMPUTER SCIENCE")` will return the same results.

Part B: Descriptive Statistics by Attribute (20 Points)

Required method (description):

- **`compute_exam_statistics_by_attribute`**

  This method computes and returns the *mean*, *median*, *mode*, and *standard deviation* of **exam scores** for each group within a specified categorical attribute. You may only group by one of the following attributes: "major", "part_time_job", and "study_environment".

  Parameters: The name of the categorical attribute to group by (i.e., "major", "part_time_job", and "study_environment")

Return: A dictionary where each **key** is a distinct value of the specified attribute, and each **corresponding value** is a dictionary summarizing that group's exam scores with keys "mean", "median", "mode", and "std_dev"

Example: Calling **`compute_exam_statistics_by_attribute("major")`** might return a dictionary similar to the one shown below. This is a view of the returned dictionary object; no output is printed to the screen.

```
{
'Mathematics': { 'mean': 77.7, 'median': 78, 'mode': 75.5, 'std_dev': 2.5 },
'Physics': { 'mean': 92.5, 'median': 90, 'mode': 91, 'std_dev': 1.2 },
'Computer Science': { 'mean': 87.5, 'median': 87.5, 'mode': 85, 'std_dev': 3.5 }
}
```

This output displays the average (mean), middle value (median), most frequent value (mode), and standard deviation of exam scores grouped by students' majors. The mean and standard deviation are always rounded to one decimal place using Python's built-in `round(value, 1)` method. The median is rounded to one decimal place only when it is the average of two middle values; if the dataset has an odd number of values, the median is returned as-is. The mode is returned exactly as it appears in the dataset without any rounding. Please ensure you follow this exact rounding approach to pass the automated tests.

- To implement this method, you should reuse the `filter_by_attribute` method defined in Task 3 – Part A to retrieve student groups for the allowed attributes before calculating the statistics.
- The method only supports the specified categorical attributes (i.e., `"major"`, `"part_time_job"`, and `"study_environment"`). If an invalid attribute is provided (e.g., "age"), the method should raise an error. For example, calling `compute_exam_statistics_by_attribute("age")` should raise the error:

```
Error: Invalid field 'age'. This method only supports specific categorical fields.
```

The output must exactly match this format for automated testing. The field name inside the error message (e.g., 'age') may change depending on the invalid attribute used, but the wording and formatting of the error message must remain the same.

Part C: Identify Top Performing Group (5 Points)

Required method (description):

- **`get_top_group_by_exam_score`**

  This method identifies the group within a specified categorical attribute that has the highest average exam score. It builds upon `compute_exam_statistics_by_attribute` method to promote modularity and code reuse.

  Parameters: The name of the categorical attribute to group by (i.e., "`major`", "`part_time_job`", "`study_environment`")

  Return: A tuple containing two elements: the group name with the highest average exam score and the corresponding average score (rounded to one decimal place)

- **Note:** The method only supports the specified categorical attributes (i.e., "`major`", "`part_time_job`", "`study_environment`") and raises an error for invalid input as defined in the previous part.
- Example: Calling **`get_top_group_by_exam_score("major")`** might return:

  ```
  ( 'Physics', 92.5 )
  ```

  This output indicates that the "Physics" group has the highest average exam score of 92.5 among all groups within the "major" attribute. The actual result may vary depending on the dataset used. The accompanying picture shows the output generated when running this method on the dataset from the previous assignment, in which *Physics* achieved the highest mean score.

Part D: Advanced Comparative Analysis (5 Points)

Required method (description):

- **`compare_groups_by_exam_score`**

  This method compares the exam performance of two specific groups (e.g., "`Library`" vs. "`Quiet Room`") within a categorical attribute (e.g., "`study_environment`"), providing structured statistical insights for each. This side-by-side comparison supports meaningful conclusions in educational or behavioral analysis.

Parameters:

- attribute: The categorical attribute used to group students (i.e., "`major`", "`part_time_job`", "`study_environment`").
- group1: The first group value to compare (e.g., "`Library`").
- group2: The second group value to compare (e.g., "`Quiet Room`").

Return: A dictionary containing:

- Separate statistics for each group, including "mean", "median", "mode", and "std_dev" of exam scores.
- The absolute difference between the two group means under the key "mean_difference".
- A field "higher_average_group" indicating which group had the higher average score.

- The method **must reuse** the method `compute_exam_statistics_by_attribute`.
- Case-insensitive matching should be used for group names.
- If a specified group is not found within the given attribute (for example, "InvalidOption" under "`study_environment`"), you should provide a clear and informative error message. Unlike previous tasks, there is no required error message format, so you have flexibility in how you communicate this issue.
- Example: Calling **`compare_groups_by_exam_score("study_environment", "Library", "Quiet Room")`** might return:

```
{
'Library': {'mean': 80.1, 'median': 80.2, 'mode': 85.5, 'std_dev': 9.2},
'Quiet Room': {'mean': 79.0, 'median': 79.0, 'mode': 80, 'std_dev': 1.4},
'mean_difference': 1.1,
'higher_average_group': 'Library'
}
```

This output presents a detailed comparison of exam scores between students who study in the "Library" and those in the "Quiet Room", based on the `study_environment` attribute. It reveals that students in the Library group achieved a higher average exam score than those in the Quiet Room group, with a mean difference of 1.1 points.

Note: To ensure your implementation is correct, you must create a comprehensive test suite for Task 3. A sample test file, `test_performance_analyzer.py`, has been provided for this class.

Task 4: Own Analysis (25 Points)

In Task 3, you explored exam scores by analyzing individual attributes, such as `major`, `part_time_job`, and `study_environment`. Now, it's your turn to take the lead. In this task, you are required to design and conduct an original analysis that uncovers **new insights about student performance**. Your goal is to go beyond simple average comparisons and address a meaningful question using statistical or exploratory techniques.

This task encourages creativity and critical thinking. You should aim to discover patterns or relationships that could inform how students learn or perform better.

Guidelines:

- Avoid repeating previous Tasks. You are expected to **design your approach and, if needed, conduct independent research that goes beyond the scope of the course material**. Students are encouraged to explore this independently using online resources and tutorials to help identify suitable analyses and interpret the results, leading to new insights.
- Your analysis should be explained clearly, and all code should be well-documented to ensure that your findings are transparent, reproducible, and easily understood by others. Clear explanations and documentation are essential for validating your approach.
- All implemented functions and analyses should be thoroughly tested to ensure correctness and reliability.
- You may use third-party Python libraries such as `pandas`, `numpy`, or `scipy` to support your analysis; this is *optional* and depends on your approach and preferences.
- If you need ideas, consider these options, but we encourage you to pursue your original question or insight if you have one.
    - **Multi-Factor comparison**: Analyze how combinations of two or more attributes (for example, `learning_style` and `study_environment`) together influence exam performance. Instead of looking at each attribute separately, examine how specific pairs or groups of characteristics correlate with higher or lower exam scores. This approach can reveal interactions or patterns that are not apparent when considering each factor alone.
    - **Motivation and performance:** Explore the relationship between students' motivation levels and their exam scores using statistical correlation techniques (e.g., Pearson or

Spearman). This can help determine whether higher motivation is associated with better exam scores.

Task 5: Interactive Visualization with Dash (Bonus: 10 Points)

This optional bonus task challenges you to create an interactive data visualization dashboard using Dash, a powerful Python framework for building analytical web applications. The primary goal is to design a clear, intuitive interface that enables users to explore exam performance trends among students by grouping data and generating visual and statistical summaries.

You may use third-party Python libraries, such as `pandas` or `numpy`; this is *optional* and depends on your approach and preferences. However, to calculate the statistical information displayed in the dashboard, you must use the functions you implemented in Tasks 2 to 4.

Note: Dash is **not covered in the course**, and completing this task will require **independent research**. You may refer to the official Dash documentation and other online tutorials.

Part A: Exam Score Distribution Visualization (5 Points)

Develop an interactive component that displays the distribution of exam scores using box plots, categorized by different student attributes (see the top part of Figure 1 and Figure 2).

What you should implement:

- Provide a dropdown menu that allows users to select a categorical attribute for grouping. The program supports only three attributes: `major`, `part-time job`, and `study environment`.
- Upon selection, the dashboard should generate a box plot that visualizes the distribution of exam scores across the corresponding categories.
- The plot must update **dynamically** based on user input.
- Ensure the plot includes clear axis labels, an informative title, and visually distinct groups to enhance interpretability.

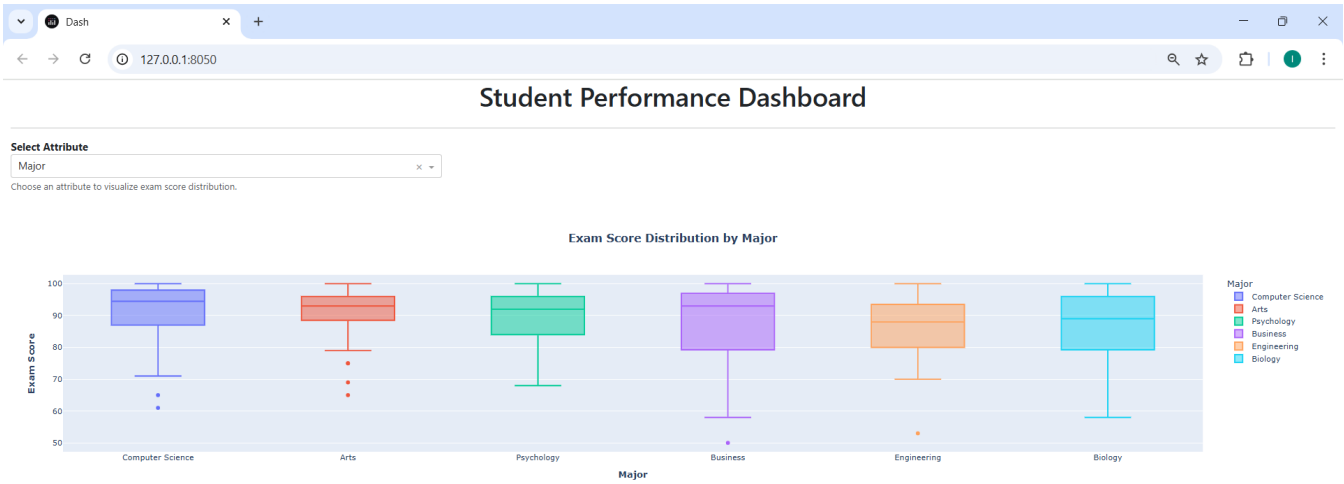
Part B: Summary Statistics Table (5 Points)

Augment the dashboard by adding a summary table that displays essential statistical metrics for each group selected in the dropdown (see the bottom part of Figure 1 and Figure 2).

What you should implement:

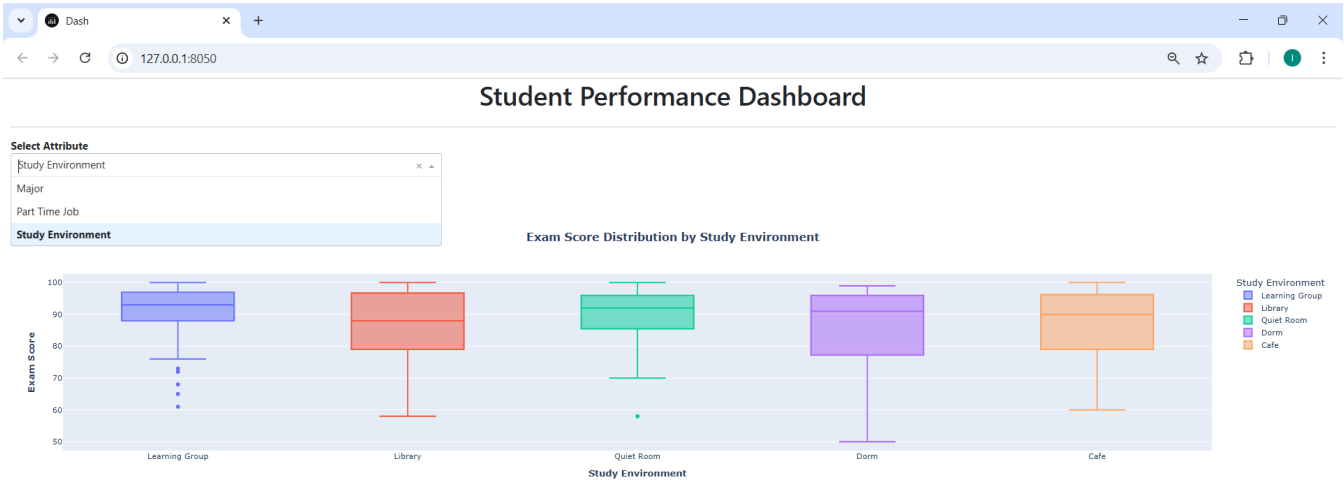
- A responsive data table positioned below the box plot.

- The table should display the **mean**, **median**, **mode**, and **standard deviation** of exam scores for each group within the selected category.
- The summary updates **automatically** based on the user's selection.



Group	Mean	Median	Mode	Std Dev
Computer Science	90.2	94.5	100	10.1
Business	87.9	93	93	11.6
Engineering	85.9	88	100	11.1
Biology	86.1	89	97	11.5
Psychology	89.2	92	98	8.6
Arts	90.6	93	96	8.6

Figure 1: Exam Score Distribution for Selecting Major



Group	Mean	Median	Mode	Std Dev
Library	85.8	88	100	11.5
Learning Group	90.8	93	93	8.7
Cafe	87	90	97	10.7
Quiet Room	89.9	92	100	9
Dorm	86.2	91	97	12.6

Figure 2: Exam Score Distribution for Selecting Study Environment

Task 6: Reflection (5 Points)

As you worked through your project, you likely encountered bugs or unexpected behavior that tested your problem-solving skills. In this task, reflect on your debugging experience by describing two meaningful issues you faced. For each one, explain:

1. **What the issue was** – Describe the bug or error clearly.
2. **How you diagnosed it** – Share how you investigated and identified the root cause.
3. **How you resolved it** – Describe the solution you applied and how it fixed the problem.

Please limit your reflection to a maximum of one page (approximately 400-500 words). This length ensures your responses are concise yet detailed enough to demonstrate your technical understanding and debugging approach.

This reflection will be **evaluated subjectively** by the instructors to assess your technical understanding and debugging approach.

Appendix: Definitions of Statistical Measures

- **Mean (Average):** The mean is the sum of all values divided by the number of values.

  Example: For the list [2, 4, 5, 7], Mean = (2 + 4 + 5 + 7) / 4 = 4.5

- **Median:** The median is the middle value in an ordered list. When the data are sorted, the median divides the list into two equal parts. If the list has an even number of values, the median is the average of the two middle numbers.

  Example: For the list [3, 1, 4], ordered as [1, 3, 4], the median is 3. For [3, 1, 4, 2], ordered as [1, 2, 3, 4], the median is (2 + 3) / 2 = 2.5

- **Mode:** The mode is the value that appears most frequently in a list. If multiple values have the same highest frequency, the first mode encountered is returned.

Example: For the list [1, 2, 3, 3, 2], both 2 and 3 appear twice, but the mode function returns 2 (the first mode).

Standard Deviation: It measures the spread or dispersion of data points around the mean. See details here: https://en.wikipedia.org/wiki/Standard_deviation